

COMMON COURSE ASSIGNMENT
CSA6501 – GENERATIVE AI & PROMPT ENGINEERING | Covering
Large Language Models & Prompt-Driven Engineering Assistance

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA6501 – Generative AI and Prompt Engineering
Academic Year / Batch	2024 - 2028
Faculty Name	Dr. Chandravathi C
Assignment Title	Design and Implementation of PromptCraft – A Prompt-Driven Engineering Assistant using Large Language Models and Prompt Engineering
Date of Issue	21-08-2026
Date of Submission	03-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO1 & CO2
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Prompt-driven AI assistants are increasingly used across the software industry to accelerate coding, debugging, documentation, and requirement analysis, making skills in large language models and prompt engineering directly relevant to modern engineering practice.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an AI-powered engineering assistant that applies core Generative AI concepts — prompt design and engineering techniques (zero-shot, few-shot, and chain-of-thought prompting), large language model (LLM) API integration, and structured evaluation of model outputs — to a unified engineering-support platform called **Prompt Craft**. Students must integrate multiple prompting strategies, build task-specific engineering modules, deploy the system as a lightweight web application, validate output quality and reliability with sample test cases, and address basic responsible-AI considerations such as transparency of prompts, hallucination handling, and reproducibility of results.

C. Problem Statement

Scenario: “PromptCraft” – A Prompt-Driven Engineering Assistant

Many students and practicing engineers find it difficult to consistently get high-quality, reliable outputs from large language models without structured prompting. Your task is to design and implement an interactive engineering-support web application called **PromptCraft** that helps engineers construct effective prompts, apply prompt engineering techniques, and use an LLM to accelerate common software-engineering tasks through a unified interface.

The system shall integrate three major functional modules:

1. Prompt Engineering Workbench

Implement zero-shot, few-shot, and chain-of-thought prompting templates for common engineering tasks.

- Users should be able to enter a task description, select a prompting strategy, and optionally supply few-shot examples or context.
- The system must display the fully constructed prompt sent to the LLM together with the generated response, enabling students to see exactly how prompt structure affects output quality.

2. LLM-Powered Engineering Task Modules

Implement at least three engineering-assistant task modules, e.g., code generation & explanation, automated debugging assistance, and requirement-to-documentation generation.

- Users will enter their engineering artifact (a code snippet, an error/bug description, or a requirement statement) as input to the selected module.
- The system should show the underlying prompt template used for each module, the LLM output, and any post-processing applied, so that students can trace how the assistant arrived at its result.

3. Prompt Strategy Evaluation & Comparison

Implement comparison of at least three prompting strategies (e.g., zero-shot, few-shot, chain-of-thought) for the same task.

- Users will enter a representative engineering task and a set of reference/expected outputs for evaluation.
- The system should display response quality, response time / latency, and token usage for each strategy, allowing clear comparative analysis.

Design the application workflow so that user inputs (task descriptions, code snippets, bug reports, requirement statements) are processed through the corresponding prompt templates and LLM calls, and the results are presented as clear prompts, responses, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate effectiveness. Students must use a real or simulated LLM API (e.g., OpenAI, Anthropic, or an open-source model) to generate responses.

Evaluate the application under suitable test cases and analyze the quality of LLM outputs, usability of the interface, and clarity of the prompt/response traces. The completed system

should also briefly address responsible-AI considerations including transparency of prompts, hallucination and error handling, data privacy, and reproducibility of results.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for a Prompt Engineering Workbench (with visible prompt construction and response display), LLM-Powered Engineering Task Modules (code generation, debugging assistance, and documentation generation), and Prompt Strategy Evaluation (quality, latency, and token-usage comparison). Accept user inputs as well as sample data. Produce comparative tables and prompt/response traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** LLM calls must return results with acceptable latency for interactive classroom use and support clear comparative analysis across prompting strategies.
- **Technical constraint:** Use Python + Streamlit. No database or complex architecture required. Use any accessible LLM API or open-source model for prompt execution.
- **Reliability:** Prompt templates and task modules must behave consistently and reproducibly; intermediate prompts must be transparent; the system must handle malformed inputs and LLM API errors without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of prompts and model limitations, reproducibility of results, mitigation of hallucinated or misleading outputs, and adherence to responsible-AI and data-privacy practices.
- **Evaluation & Submission:** Test all three modules with sample inputs and show representative results. Submit working Streamlit code, a short report containing sample prompts/outputs / screenshots, comparative analysis, and a GitHub repository link.

E. Student Work

1. Problem Understanding and Formulation

Problem Understanding

The proposed system, PromptCraft – A Prompt-Driven Engineering Assistant, is designed to address the difficulty of obtaining reliable and task-specific assistance from Large Language Models during software-engineering activities. Although LLMs can generate code, analyse errors and prepare technical documentation, the quality of their responses is strongly influenced by the way instructions are formulated.

The proposed system therefore treats prompt construction as an engineering process rather than simply sending a user's question directly to an LLM. PromptCraft provides a structured environment where users can select an appropriate prompting strategy, construct the corresponding prompt, submit it to an LLM, inspect the generated response and evaluate the result.

The framework incorporates zero-shot, few-shot and chain-of-thought prompting and applies them to practical engineering tasks.

Challenges Identified

The major challenges addressed by the system are:

- Formulating clear and unambiguous prompts for engineering tasks.
- Selecting a suitable prompting strategy for a particular task.
- Providing examples when the desired output format is difficult to describe.
- Handling engineering inputs such as source code, errors and requirements.
- Maintaining visibility of the prompt sent to the model.
- Comparing generated responses objectively.
- Managing latency and token usage during LLM interaction.
- Identifying possible hallucinations or technically incorrect responses.
- Handling malformed inputs and model/API failures.
- Maintaining reproducibility when comparing different prompting approaches.

Expected Outcomes

The expected outcome is an interactive engineering-support platform capable of:

1. Constructing prompts using multiple prompting strategies.
2. Generating engineering-oriented responses using an LLM.
3. Supporting code generation and explanation.
4. Providing debugging assistance.

5. Converting requirements into technical documentation.
6. Comparing prompting strategies using measurable parameters.
7. Displaying prompt-response traces.
8. Handling basic errors and unreliable model outputs.

Assumptions

For implementation and evaluation, the following assumptions are made:

Parameter	Assumption
Application type	Interactive web application
Development language	Python
Interface framework	Streamlit
Prompting methods	Zero-shot, Few-shot, Chain-of-thought
Engineering modules	Code, Debugging, Documentation
Model interaction	Real or simulated LLM
Evaluation parameters	Quality, latency, token usage
Database	Not required
Test cases	2–3 representative cases per module

These assumptions keep the system sufficiently simple for implementation using standard student computing resources while still demonstrating the required Generative AI concepts.

2. Application of Course Knowledge (CO2, CO3, CO4)

Part I – Prompt Engineering and LLM Interaction

2.1 Zero-Shot Prompting

In zero-shot prompting, the model receives the engineering task directly without any demonstration examples.

General structure:

Role/Context

+

Engineering Task

+

Expected Output Instruction

For example: You are a software engineering assistant.

Explain the purpose of exception handling in Python and provide a suitable example.

This method has a simple prompt structure and is suitable for clearly defined tasks.

2.2 Few-Shot Prompting

Few-shot prompting provides examples along with the new task. The examples demonstrate the expected response behaviour.

General structure:

Role/Context

+

Example 1

+

Example 2

+

New Engineering Task

+

Output Instruction

The examples can guide the model regarding terminology, structure, explanation style or expected output format.

2.3 Chain-of-Thought Prompting

Chain-of-thought prompting is used for engineering problems that require multiple reasoning stages. Instead of requesting only a direct answer, the prompt encourages systematic analysis of the problem before producing the final response.

It is particularly applicable to debugging and engineering analysis where the model must consider several conditions before proposing a solution.

2.4 Engineering Task Modules

PromptCraft applies these techniques to three engineering modules:

Module 1 – Code Generation and Explanation

The module accepts a programming requirement and generates an appropriate implementation with an explanation of its operation.

Module 2 – Automated Debugging Assistance

The module accepts source code, an error message, or a description of unexpected behaviour and provides a structured debugging analysis.

Module 3 – Requirement-to-Documentation Generation

The module accepts software requirements and transforms them into structured technical documentation containing objectives, functional requirements and expected behaviour.

3. Solution / Design / Methodology

The proposed architecture follows a modular workflow:

User Input → Module Selection → Prompt Strategy Selection → Prompt Construction → LLM Processing → Response Processing → Evaluation → Result Display

Stage 1 – User Input

The user enters an engineering task, source code, error description or software requirement.

Stage 2 – Module Selection

The system identifies the required engineering operation:

- Code generation
- Debugging assistance
- Documentation generation

Stage 3 – Prompt Strategy Selection

The user selects zero-shot, few-shot or chain-of-thought prompting.

Stage 4 – Prompt Construction

PromptCraft combines the user's input with the selected template. In few-shot mode, examples are inserted into the prompt. Additional instructions can be included to control the expected response.

Stage 5 – LLM Processing

The constructed prompt is sent to a real or simulated LLM. The generated response is captured by the application.

Stage 6 – Evaluation

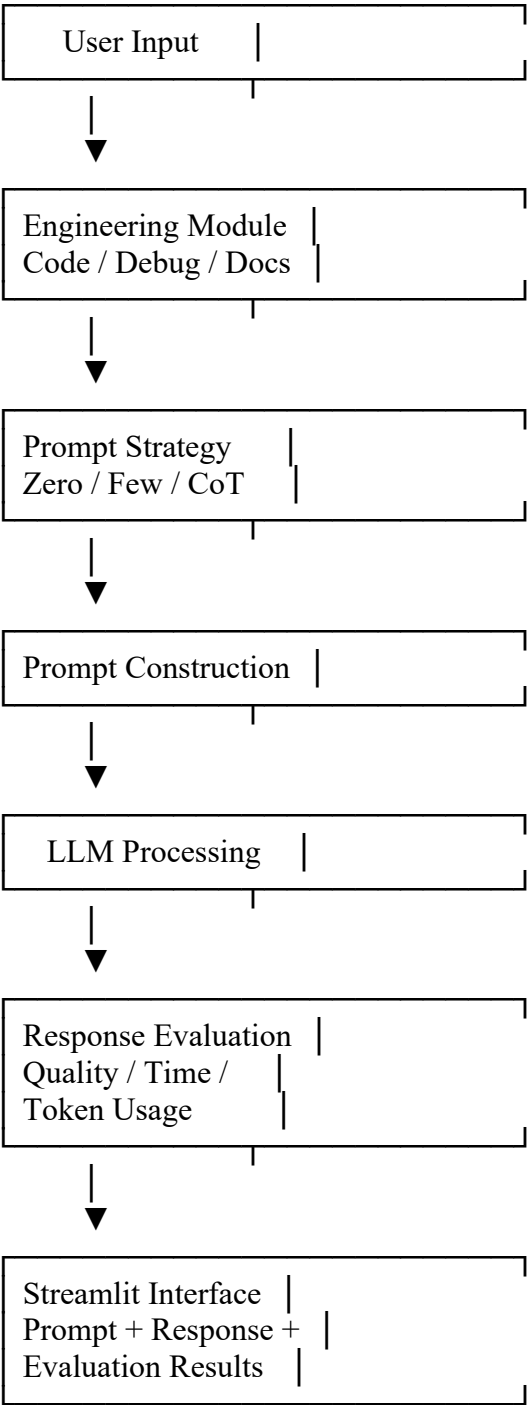
The generated response is evaluated using:

- Response quality
- Response latency
- Token usage

Stage 7 – Presentation

The application displays the constructed prompt, generated response and evaluation information through the Streamlit interface.

Proposed Architecture



4. Use of Modern Tools

The proposed system uses modern Generative AI development tools to implement and demonstrate the PromptCraft framework.

Python

Python is used for application logic, prompt construction, response processing and evaluation.

Streamlit

Streamlit is used to develop the interactive web interface. It allows users to select modules, enter engineering tasks, choose prompting strategies and view results without requiring a complex frontend framework.

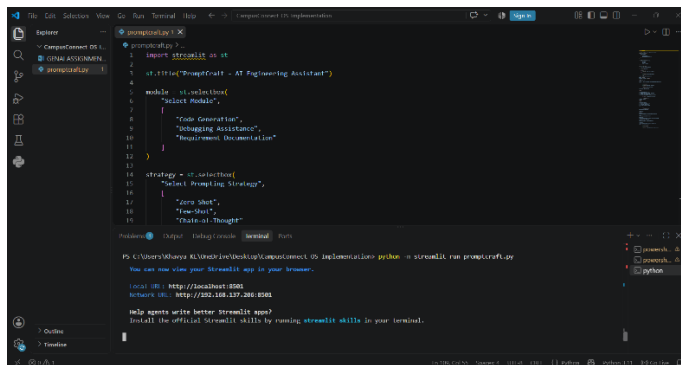
LLM / Simulated LLM

A real LLM API or simulated model can be used to generate engineering responses. The architecture separates prompt construction from model execution, allowing the model component to be replaced without redesigning the complete application.

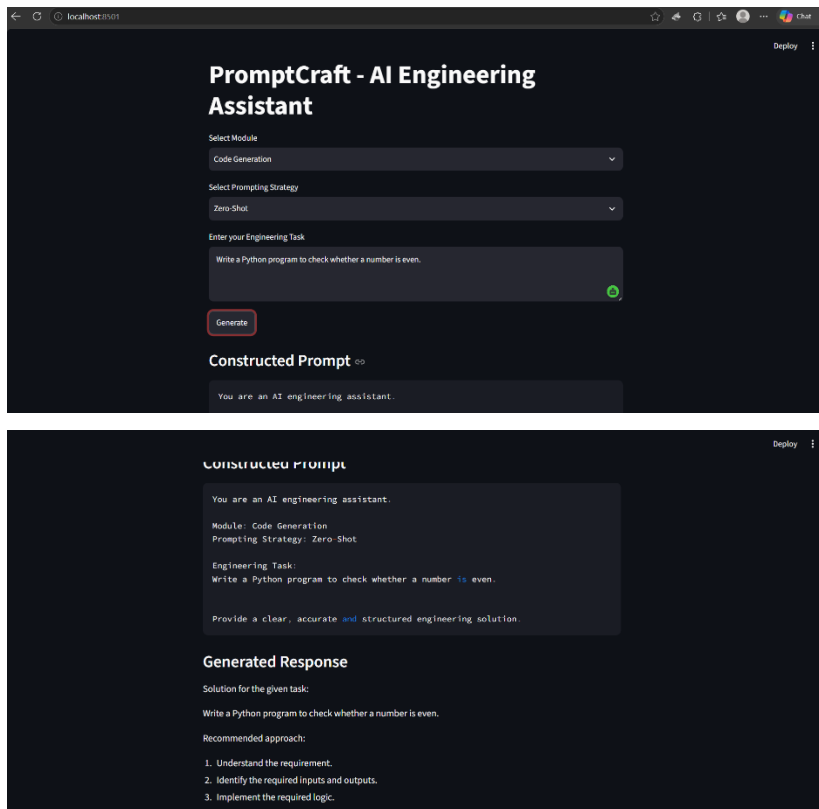
VS Code

VS Code is used as the development and execution environment for the Python application.

Implementation Evidence:



Prompt Execution Evidence:



5. Results and Validation

The system is validated using representative test cases for each engineering module.

Code Generation Validation

A programming requirement is provided as input and processed using the selected prompting strategy. The generated code and explanation are examined for relevance, completeness and alignment with the requirement.

Debugging Validation

A representative code error or debugging scenario is provided. The generated response is checked for identification of the likely cause, explanation of the problem and appropriateness of the suggested correction.

Documentation Validation

A software requirement is provided to the documentation module. The generated output is checked for proper organization, coverage of the requirement and technical clarity.

Prompt Strategy Validation

The same task is submitted using zero-shot, few-shot and chain-of-thought strategies. Their outputs are compared using:

Metric	Purpose
Response Quality	Measures relevance and completeness
Latency	Measures response-generation time
Token Usage	Estimates prompt and response size

Validation Evidence

The following evidence will be added after implementation:

- Prompt construction screenshot
 - Generated response screenshot
 - Code-generation output
 - Debugging output
 - Documentation output
 - Strategy comparison table
 - Quality comparison graph
 - Token usage comparison graph
-

6. Analysis and Engineering Decision

The comparison of prompting methods indicates that no single strategy is universally optimal for every engineering task.

Zero-shot prompting has a relatively simple structure and is suitable for direct and well-defined tasks. It reduces the need for additional examples and can therefore be efficient when the desired response is straightforward.

Few-shot prompting introduces examples that guide the model toward a particular response structure. This is useful when consistency and formatting are important. The additional examples, however, increase the amount of prompt information supplied to the model.

Chain-of-thought prompting is more appropriate for tasks requiring structured analysis. Debugging and complex engineering questions can benefit from systematic reasoning instructions, although the resulting interaction may require more processing.

Based on these characteristics, PromptCraft adopts a multi-strategy design rather than enforcing one prompting method for every task.

The engineering decision is therefore:

- Use **zero-shot** for direct engineering questions.
 - Use **few-shot** when examples can improve response consistency.
 - Use **chain-of-thought** for tasks requiring multi-stage analysis.
 - Use task-specific prompt templates for code, debugging and documentation.
 - Evaluate strategies quantitatively rather than relying only on subjective judgement.
-

7. Broader Considerations

Reliability and Safety

LLM responses cannot automatically be assumed to be technically correct. Generated code, debugging recommendations and documentation should therefore be reviewed before practical use.

Sustainability and Resource Efficiency

Efficient prompt construction can reduce unnecessary model input and output. Avoiding unnecessarily long prompts can reduce token consumption and computational requirements, particularly when the system is used repeatedly.

Accessibility and Usability

The Streamlit interface provides a straightforward environment in which users can experiment with different prompting strategies without directly implementing API communication or complex frontend components.

Ethics and Professional Responsibility

Engineering assistants should support human decision-making rather than replace professional verification. Users must be aware of model limitations and should validate generated technical information.

Privacy

Engineering artifacts may contain proprietary code, project requirements or other sensitive information. The system should avoid unnecessary collection or transmission of confidential data, particularly when an external LLM API is used.

Transparency

Displaying the constructed prompt and generated response improves transparency by allowing users to understand how the model was instructed and how the response was produced.

8. Conclusion

PromptCraft presents a structured approach to applying Generative AI within software-engineering workflows. The system combines prompt engineering, LLM interaction, engineering-specific task modules and quantitative evaluation into a single Streamlit-based application.

The proposed framework supports zero-shot, few-shot and chain-of-thought prompting and applies these techniques to code generation, debugging assistance and requirement-to-documentation generation. The visible prompt-response workflow improves transparency, while quality, latency and token measurements provide a basis for comparing different prompting strategies.

The system also incorporates responsible-AI considerations including hallucination awareness, human validation, privacy, error handling and reproducibility. The modular architecture allows additional engineering tasks, LLM providers and evaluation techniques to be incorporated in future versions.

9. Student Reflection

The development of PromptCraft provides practical understanding of how Generative AI concepts can be transformed into an engineering application. One of the major observations is that the quality of an LLM response is influenced not only by the model itself but also by the structure and information contained in the prompt.

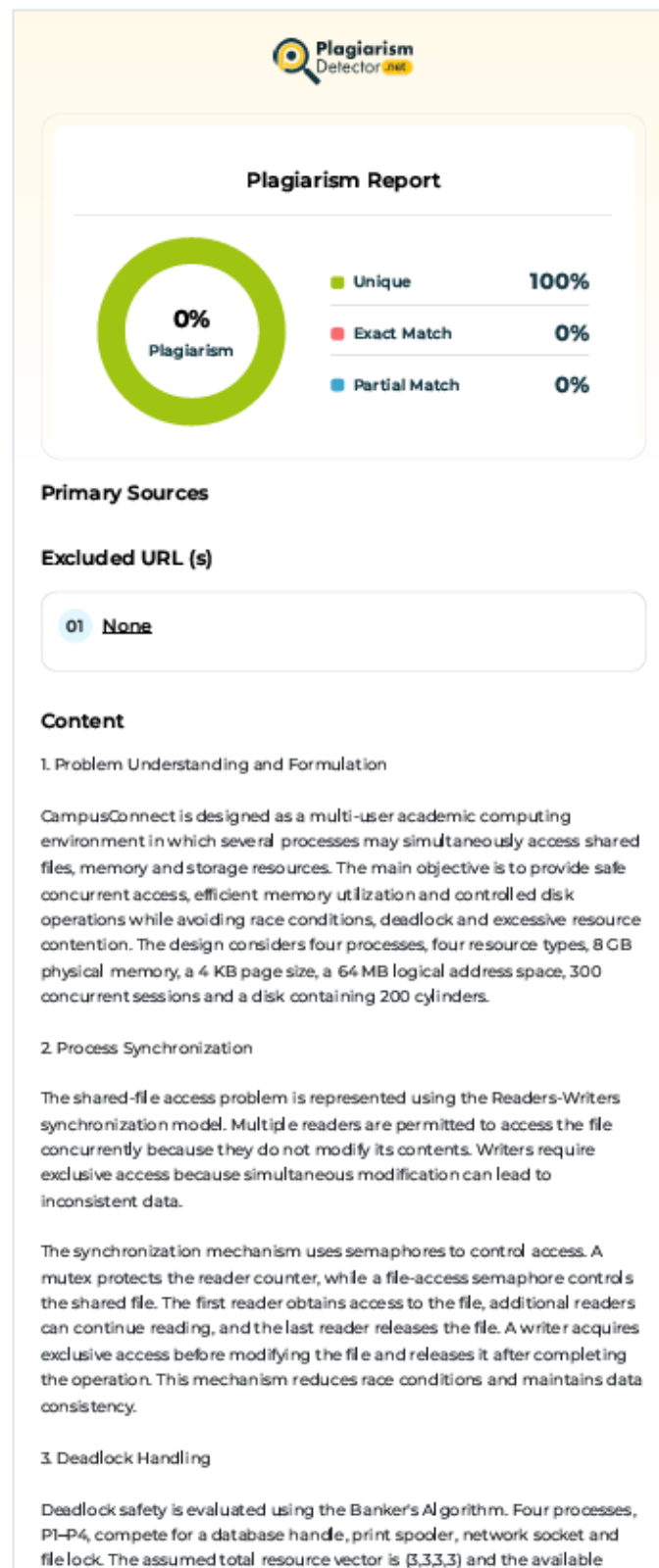
Working with zero-shot, few-shot and chain-of-thought strategies demonstrates the importance of selecting a prompting approach according to the task. The project also highlights the value of task-specific prompt templates instead of relying on a single generic instruction.

If additional resources were available, the system could be improved by integrating multiple LLM providers, adding automated semantic evaluation, supporting additional engineering tasks, maintaining experiment histories and testing the system with larger and more diverse engineering datasets.

10. References

- [1] T. Brown et al., “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [2] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [3] A. Vaswani et al., “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [4] Python Software Foundation, *Python Documentation*.
- [5] Streamlit, *Streamlit Documentation*.
- [6] Documentation of the LLM/API selected for the PromptCraft implementation.
- [7] Course notes and laboratory materials provided for the Generative AI course.

PLAGIARISM REPORT:



F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Prompt Engineering, LLM Integration, Evaluation)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Python, LLM APIs, Prompt Engineering Frameworks)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Prompt Engineering Techniques	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – LLM Task Module Integration	CO1	PO1, PO2, PO3	L3/L4	6
Application – Evaluation & Comparison of Prompts	CO2	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated PromptCraft	CO1–CO2	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage (Python, LLM APIs, Prompt Engineering)	CO1–CO2	PO5	L3	10
Validation of results	CO1–CO2	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO2	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO2	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA08 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement: _____

Corrective / Improvement Action:

Action to be implemented in the next offering:
